

RL78/78K0R マクロコレクション

本冊子は RL78/78K0R

用マクロ集の概要と、収録一覧（全95マクロ）を掲載したパンフレットです。

◆ 特長（抜粋）

- ・ フラグ非依存設計：CMP 後の CY/Z 差異をマクロ内部で吸収し、分岐の一貫性を確保。
- ・ 自己代入エラー回避：ACC 別名等で MOV A,A の混乱を防止。
- ・ 条件付きストア命令：if を書かずに条件代入を実現。
- ・ 高級言語ライク：範囲判定・多分岐・ループを 1 行で記述。

◆ C 言語との対応例

if (10 <= x && x <= 20) → BIRANGE x,#10,#20,label

while(--cnt) → BDNZ cnt,label

switch(cmd){...} → TBLBR cmd,table

memset(buf,0,16) → FILL buf,#0,#16

if (x==0) return; → RETZ

◆ ライセンス概要（有料版）

個人利用：自由

教育機関：公教育は無償、営利教育は契約要

商用利用：自社開発案件で利用可（再販禁止）

再配布禁止：マクロ本体・文書の再配布不可

◆ 収録マクロ一覧（全95）

01. LDAT — LDAT cond1, srcb — LoaD register A if True

02. LDAF — LDAF cond1, srcb — LoaD register A if False

03. STAT — STAT cond1, dstb — STore register A if True

04. STAF — STAF cond1, dstb — STore register A if False

05. MOVMM — MOVMM dstb, srcb — MOVe Memory to Memory*

06. MOVXM — MOVXM srcb — MOVe Memory to X-reg

07. MOVMMT — MOVMMT cond1, dstb, srcb — MOVe Memory to Memory if True

08. MOVMMF — MOVMMF cond1, dstb, srcb — MOVe Memory to Memory if False

09. MOVLF — MOVLF dstb, logic, srcb — MOVe via Logic Filter

10. OVWMIX — OVWMIX dstb, srcb, bitpt — MIXing byte data (OVer Witte)

11. ORMIX — ORMIX dstb,srcb,bitpt — MIXing byte data (OR)
12. ANDMIX — ANDMIX dstb,srcb,bitpt — MIXing byte data (AND)
13. XORMIX — XORMIX dstb,srcb,bitpt — MIXing byte data (Exclusive OR)
14. MOVWMM — MOVWMM dstw,srcw — MOVe Word Memory to Memory*
15. COM — COM prb — Complement (1's complement) *
16. NEG — NEG prb — Negate (2's complement) *
17. SWAPM — SWAPM prb — SWAP nibbles of data memory*
18. SWAPA — SWAPA — SWAP nibbles of register A
19. ADDXA — ADDXA — ADD register X to register A*
20. ORW_AX — ORW_AX prw — OR Word on pair register AX
21. ANDW_AX — ANDW_AX prw — AND Word on pair register AX *
22. XORW_AX — XORW_AX prw — Exclusive OR Word on pair register AX *
23. SKIZ — SKIZ prb — SKip if Increment data memory is Zero*
24. SKDZ — SKDZ prb — SKip if Decrement data memory is Zero*
25. BDNZ — BDNZ prb,addr — Branch if Decrement data memory is Not Zero*
26. RETDZ — RETDZ prb — RETurn from subroutine if Decrement data memory is Zero*
27. RETDNZ — RETDNZ prb — RETurn from subroutine if Decrement data memory is Not Zero**
28. RETDNZW — RETDNZW prw — RETurn from subroutine if Decrement data memory is Zero* (16bit)
29. RETDZW — RETDZW prw1 — RETurn from subroutine if Decrement data memory is Not Zero (16bit)
30. RORWC_AX — RORWC_AX — Rotate Right Word with Carry on AX *
31. MOV1MM — MOV1MM ds1, sr1 — MOVe bit Memory to Memory*
32. MOV1RM — MOV1RM ds1, sr1 — ReVersal MOVe bit Memory to Memory
33. REV1 — REV1 pr1 — ReVersal Bit*
34. BITBUF — BITBUF prb,pr1 — BIT BUFfring
35. BITBUFW — BITBUFW prw,pr1 — BIT BUFfring (16bit)
36. TBLBR — TBLBR top_addr,pnt — TabLe Branch
37. BEQA — BEQA prb,addr — Branch If EQual register A
38. BEQ — BEQ prb1,prb2,addr — Branch if EQual *
39. BNE — BNE prb1,prb2,addr — Branch if Not Equal *
40. BGT — BGT prb1,prb2,addr — Branch if Greater Than (prb1 > prb2)
41. BGE — BGE prb1,prb2,addr — Branch if Greater Than or Equal (prb1 >= prb2)

42. BLT — BLT prb1, prb2, addr — Branch if Less Than ($\text{prb1} < \text{prb2}$)
43. BLE — BLE prb1, prb2, addr — Branch if Less Than or Equal ($\text{prb1} \leq \text{prb2}$)
44. BBC — BBC pr1, addr — Branch if Bit Clear *
45. BBS — BBS pr1, addr — Branch if Bit Set *
46. WAITBS — WAITBS bit — WAITing until Bit Set
47. WAITBC — WAITBC bit — WAITing until Bit clear
48. WAITCZ — WAITCZ prb — WAITing until Count Zero
49. SKT — SKT pr1 — SKip if True *
50. SKF — SKF pr1 — SKip if False *
51. SKEQ — SKEQ prb1, prb2 — SKip if EQual *
52. SKNE — SKNE prb1, prb2 — SKip if Not Equal *
53. SKGT — SKGT prb1, prb2 — SKip if Greater than Zero *
54. SKGE — SKGE prb1, prb2 — SKip if Greater than or Equal to Zero *
55. SKLT — SKLT prb1, prb2 — SKip if Less than Zero *
56. SKLE — SKLE prb1, prb2 — SKip if Less than or Equal to Zero *
57. TBCALL — TBCALL top_addr, pnt — TabeL subroutine CALL *
58. CALL_F — CALL_F pr1, addr — subroutine CALL If False *
59. CALL_T — CALL_T pr1, addr — subroutine CALL If True *
60. CALLEQA — CALLEQA prb, addr — subroutine CALL If EQual register A
61. RETF — RETF pr1 — RETurn from subroutine If False *
62. RETT — RETT pr1 — RETurn from subroutine If True *
63. RETZ — RETZ — RETurn from subroutine if Zero ($Z = 1$)
64. RETNZ — RETNZ — RETurn from subroutine if Not Zero ($Z = 0$)
65. RETC — RETC — RETurn from subroutine if set carry flag *
66. RETNC — RETNC — RETurn from subroutine if clear carry flag *
67. RETEQ — RETEQ prb1, prb2 — ReTurn if EQual *
68. RETNE — RETNE prb1, prb2 — ReTurn if Not Equal *
69. RETEQZ — RETEQZ prb — ReTurn if EQual Zero
70. RETNEZ — RETNEZ prb — ReTurn if Not EQual Zero
71. RETEQZW — RETEQZW prw — ReTurn if EQual Zero (16bit)
72. RETNEZW — RETNEZW prw — ReTurn if Not EQual Zero (16bit)
73. RET_A — RET_A imd — RETurn from subroutine and load immediate data to register A

- 74. STACK_SAVE — STACK_SAVE — When an interrupt occurs, the PSW is automatically saved.
- 75. STACK_LOAD — STACK_LOAD — When returning from an interrupt, the PSW is automatically restored.
- 76. PICKUP — PICKUP top_addr, pnt — PICK UP from tabble to register A
- 77. PICKUPW — PICKUPW top_addr, pnt — PICK UP from tabble to pair register AX
- 78. PUTB — PUTB top_addr, pnt — PUT data of register A to TABble (buffer)
- 79. PUTW — PUTW top_addr, pnt — PUT data of pair register to TABble (buffer)
- 80. BLKMOV — BLKMOV dst_addr, src_addr, cnt — BLock_MOVE *
- 81. FILL — FILL top_addr, cnt, dat — FILL in the memory *
- 82. TSTM — TSTM prb — zero TeST Memory *
- 83. TSTA — TSTA — zero TeST register A*
- 84. TSTAX — TSTAX — zero TeST pair register AX *
- 85. TSTB — TSTB patn, mem — TeST Bit *
- 86. CMBT — CMBT patn, mem — CoMpare BiT pattern*
- 87. RANGE — RANGE prb1, prb2, prb3 — test in Range
- 88. BIRANGE — BIRANGE prb1, prb2, prb3, addr — Branch if In RANGE
- 89. BORANGE — BORANGE prb1, prb2, prb3, addr — Branch if Out RANGE
- 90. SIRANGE — SIRANGE prb1, prb2, prb3 — Skip if In RANGE
- 91. SORANGE — SORANGE prb1, prb2, prb3 — Skip if Out RANGE
- 92. INCWZ — INCWZ prw — INCrement memory with result z
- 93. DECWZ — DECWZ prw — DECrement memory with result z
- 94. MOVQMI — MOVQMI prq, immq — MOVE Quad byte immediate to Memory*
- 95. DECQZ — DECQZ prq — Decrement Quad bytes(32bit), set Z if result == 0

販売場所： https://note.com/lopra_lab

(※販売ページでは無料公開部分を閲覧いただけます)

価格： 1,000円 (税込)

お問い合わせ： <https://www.lowpowerradiolab.org/contactus/>



Low-Power Radio Research & Applications Laboratory (LoPRA Lab)

<https://www.lowpowerradiolab.org/>